

Cybersecurity Incident Report Generator

Project submission — working MVP, source code, security implementation, and test results

PROJECT	Cybersecurity Incident Report Generator
STACK	Python 3.12 · Flask · gunicorn · reportlab
REPOSITORY	github.com/leopardz0ne/incident-report-generator
TEST STATUS	31 / 31 automated tests passing

31 / 31 TESTS PASSING

1

Overview

This project generates standardized, tamper-evident cybersecurity incident reports (Markdown and PDF) from investigation data, through a web interface or a JSON API. It is built for a SOC or incident response team to turn raw investigation notes — timeline entries, indicators of compromise, containment steps — into a consistent, professional report every time.

Core features

- Console-style web UI with a live status bar (clock, severity/TLP badges), sample-data loader, and a client-side history of recent reports
- JSON API (`/api/generate`, `/api/generate_pdf`) for integration with a SIEM or IR playbook automation
- Standardized report structure: metadata, scope, description, timeline, IOCs, impact, containment/eradication/recovery, root cause, recommendations
- Download menu offering PDF, Markdown, or browser print
- SHA-256 integrity hash stamped on every generated report

2

Working project (MVP)

The application was verified end-to-end under its real production endpoint (gunicorn, 2 workers — the same command used in the Docker image and on the hosting platform), not just the Flask development server. The following were confirmed working immediately before this document was generated:

CHECK	RESULT
Index page (GET /)	✓ 200 OK
Static assets (CSS / JS)	✓ 200 OK
Security headers present	✓ X-Frame-Options, X-Content-Type-Options, Content-Security-Policy
Report generation (POST /api/generate)	✓ 200 OK — valid Markdown + SHA-256 hash returned
PDF generation (POST /api/generate_pdf)	✓ 200 OK — valid multi-page PDF returned

Deployment. The project ships with a Dockerfile, Procfile, render.yaml (Render.com blueprint), and fly.toml, and is deployed live on Render.com. See `DEPLOYMENT.md` in the repository for step-by-step instructions covering Render, Fly.io, and generic Docker hosts.

Full source is included in the accompanying archive and pushed to the GitHub repository listed on the cover page. Project layout:

incident-report-generator/	
app.py	Flask app — routes, security headers, rate limiting
models.py	IncidentReport data model + field constraints
security.py	Input validation, sanitization, hashing, audit log, rate limiter
report_generator.py	Builds reports, renders Markdown + PDF
templates/index.html	Web UI (console-style layout)
static/style.css	UI styling — external file, required by strict CSP
static/app.js	UI behavior — live badges, download menu, history
tests/test_report_generator.py	31 automated tests
reports/	Example generated reports
Dockerfile, Procfile, render.yaml, fly.toml	Deployment configs
requirements.txt, README.md, DEPLOYMENT.md	Docs and dependencies

FEATURE	WHERE	PURPOSE
Allow-list field validation	security.py	Severity / status / classification checked against fixed enums, not accepted as free text
Input sanitization (HTML-escaping)	security.py: <code>sanitize_text()</code>	Every free-text field is escaped and stripped of control characters before storage / render
Restricted incident ID charset	security.py	incident_id must match <code>^[A-Za-z0-9_-]+\$</code> , closing injection via a field used in filenames / URLs
Field length limits	models.py, security.py	Caps payload size, guards against resource-exhaustion via oversized input
Request body size cap	app.py (512 KB)	Rejects oversized requests before they are even parsed
Rate limiting	security.py: <code>RateLimiter</code>	Sliding-window limiter (20 req/min/client) on report-generation endpoints
Strict security headers	app.py	CSP with no unsafe-inline, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy
No hardcoded secrets	app.py	SECRET_KEY read from environment; random per-process fallback, never a shipped default
Report integrity hashing	security.py: <code>compute / verify_integrity_hash()</code>	SHA-256 hash on every report; constant-time comparison for verification
Fail-closed error handling	app.py error handlers	Internal errors return a generic message; details go only to the server-side audit log
Audit logging	security.py: <code>audit_log()</code>	Generation, validation failures, and rate-limit hits logged, excluding raw free text
Debug mode disabled by default	app.py	Flask's interactive debugger is off unless <code>FLASK_DEBUG=1</code> is explicitly set
No unsafe deserialization / eval	throughout	Only Flask's built-in JSON parser is used — no eval, exec, pickle, or <code>yaml.load</code> on input
Non-root container user	Dockerfile	Container runs as an unprivileged user, limiting blast radius if compromised

5 Test results

31 automated tests covering field validation, XSS/injection resistance, Markdown and PDF generation, integrity hashing, rate limiting, and the Flask API endpoints. Run with `pytest tests/ -v`.

31 PASSED · 0 FAILED

```
pytest tests/ -v
```

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/incident-report-generator
collecting ... collected 31 items

tests/test_report_generator.py::TestValidation::test_valid_payload_passes PASSED [ 3%]
tests/test_report_generator.py::TestValidation::test_missing_required_field_rejected PASSED [ 6%]
tests/test_report_generator.py::TestValidation::test_invalid_severity_rejected PASSED [ 9%]
tests/test_report_generator.py::TestValidation::test_invalid_status_rejected PASSED [ 12%]
tests/test_report_generator.py::TestValidation::test_invalid_classification_rejected PASSED [ 16%]
tests/test_report_generator.py::TestValidation::test_bad_incident_id_charset_rejected PASSED [ 19%]
tests/test_report_generator.py::TestValidation::test_bad_date_format_rejected PASSED [ 22%]
tests/test_report_generator.py::TestValidation::test_oversized_field_rejected PASSED [ 25%]
tests/test_report_generator.py::TestValidation::test_non_dict_payload_rejected PASSED [ 29%]
tests/test_report_generator.py::TestValidation::test_timeline_must_be_list PASSED [ 32%]
tests/test_report_generator.py::TestValidation::test_too_many_timeline_events_truncated_with_error PASSED [ 35%]
tests/test_report_generator.py::TestValidation::test_iocs_must_be_list PASSED [ 38%]
tests/test_report_generator.py::TestSanitization::test_script_tag_escaped PASSED [ 41%]
tests/test_report_generator.py::TestSanitization::test_control_characters_stripped PASSED [ 45%]
tests/test_report_generator.py::TestSanitization::test_full_payload_with_xss_attempt_is_neutralized PASSED [ 48%]
tests/test_report_generator.py::TestSanitization::test_sql_like_string_in_free_text_is_just_escaped_text PASSED [ 51%]
tests/test_report_generator.py::TestReportGeneration::test_markdown_contains_key_fields PASSED [ 54%]
tests/test_report_generator.py::TestReportGeneration::test_pdf_generation_produces_nonempty_bytes PASSED [ 58%]
tests/test_report_generator.py::TestReportGeneration::test_empty_timeline_and_iocs_handled_gracefully PASSED [ 61%]
tests/test_report_generator.py::TestIntegrity::test_hash_is_deterministic PASSED [ 64%]
tests/test_report_generator.py::TestIntegrity::test_hash_changes_when_content_changes PASSED [ 67%]
tests/test_report_generator.py::TestIntegrity::test_verify_integrity_detects_tampering PASSED [ 70%]
tests/test_report_generator.py::TestRateLimiter::test_allows_up_to_limit PASSED [ 74%]
tests/test_report_generator.py::TestRateLimiter::test_blocks_beyond_limit PASSED [ 77%]
tests/test_report_generator.py::TestRateLimiter::test_different_keys_independent PASSED [ 80%]
```

```
tests/test_report_generator.py::TestApiEndpoints::test_index_loads PASSED [ 83%]
tests/test_report_generator.py::TestApiEndpoints::test_generate_valid_payload PASSED [ 87%]
tests/test_report_generator.py::TestApiEndpoints::test_generate_invalid_payload_returns_422 PASSED [ 90%]
tests/test_report_generator.py::TestApiEndpoints::test_generate_non_json_rejected PASSED [ 93%]
tests/test_report_generator.py::TestApiEndpoints::test_security_headers_present PASSED [ 96%]
tests/test_report_generator.py::TestApiEndpoints::test_pdf_endpoint_returns_pdf PASSED [100%]

===== 31 passed in 0.50s =====
```

Result: 31 passed, 0 failed. Full pytest output captured verbatim from the most recent test run.